Microservices Architecture

with Spring Boot

Complete Production-Grade Learning Guide

Scratch → Advanced • 2025 Industry Standards • Interview Ready

For Java Backend Developers (3–5 Years Experience)

Targeting Top Product-Based Companies

© 2025 — Spring Boot 3.x | Spring Cloud 2023.x | Java 21

🚀 Section 1: Introduction to Microservices

What they are, why they exist, and when NOT to use them

1.1 What are Microservices?

Microservices is an architectural style that structures an application as a collection of small, independently deployable services. Each service is focused on a single business capability, runs in its own process, and communicates via well-defined APIs.

Think of it like a city vs. a single giant building. A city has specialized buildings — hospitals, schools, banks — each with its own purpose, staff, and rules. Microservices do the same for software.

1.2 Monolith vs Microservices — Real Comparison

Dimension	Monolith	Microservices
Codebase	Single unified codebase	Multiple small, focused repos
Deployment	Deploy entire app for any change	Deploy only the changed service
Scaling	Scale entire app (costly)	Scale individual services independently
Tech Stack	Single language/framework	Polyglot (different stacks per service)
Team Structure	One large team	Small autonomous teams (2-pizza rule)
Failure Impact	One bug can crash everything	Failure is isolated to one service
Development Speed	Slows as app grows	Teams work independently in parallel
Data	Single shared database	Each service owns its own database
Complexity	Simple initially, complex later	Complex upfront infra, simpler logic
Testing	Easier integration testing	Requires contract + integration tests

1.3 Why Microservices?

- Independent deployability: Deploy user-service without touching payment-service
- Fault isolation: A crash in notification-service does not bring down checkout
- Technology flexibility: Use Node.js for real-time chat, Java for order processing
- Scalability: Black Friday sale? Scale only the order-service, not the entire platform
- Organizational alignment: One team owns one service end-to-end (Conway's Law)
- Faster time to market: Teams deploy 50+ times per day at Amazon

1.4 When NOT to Use Microservices

⚠️ COMMON MISTAKE / WARNING

Do NOT start with microservices for a new product with unproven business requirements.

Do NOT use if your team is < 5 developers. Overhead kills productivity.
Do NOT adopt if you lack DevOps maturity (Docker, K8s, CI/CD pipelines).
Do NOT use if your domain is not yet well-understood. Start monolith, extract later.
"Monolith First" — Martin Fowler recommends building a monolith, then decomposing.

1.5 Industry Use Cases

Company	Challenge Solved	Key Pattern Used
Netflix	Global streaming at scale, region failover	Circuit breaker, Chaos Engineering, Eureka
Amazon	Each team owns a full service (2-pizza rule)	Event-driven, SOA → Microservices
Uber	Geolocation, ride matching, pricing real-time	Domain partitioning, gRPC, Kafka
Airbnb	Multiple product verticals (stays, exp, etc)	GraphQL federation, service mesh
LinkedIn	Feed generation at 900M users	Async messaging, data streaming

⚡ QUICK REVISION
• Microservices = small, independent, single-responsibility services
• Each service has its own database, its own deployment, its own team
• Not a silver bullet — startups and small teams should start with monoliths
• Key benefits: scalability, fault isolation, independent deployment, team autonomy
• Conway's Law: org structure mirrors architecture — align teams with services

🧱 Section 2: Microservices Fundamentals

Decomposition, DDD, and communication patterns

2.1 Service Decomposition Strategies

The hardest part of microservices is deciding WHERE to split. Wrong cuts cause chatty services, distributed monoliths, and pain.

Strategy 1: Decompose by Business Capability

Organize services around what the business does, not how it does it.

- User Management Service → handles registration, login, profile
- Product Catalog Service → handles product listings, search, categories
- Order Service → handles cart, checkout, order lifecycle
- Payment Service → handles billing, refunds, invoices
- Notification Service → handles email, SMS, push notifications

Strategy 2: Decompose by Subdomain (DDD)

Domain-Driven Design (DDD) provides the vocabulary. A Bounded Context is a logical boundary within which a domain model is consistent.

- Core Domain: Your competitive advantage (e.g., ride-matching algorithm for Uber)
- Supporting Domain: Necessary but not unique (e.g., payment processing)
- Generic Domain: Commodity functionality (e.g., email sending via SendGrid)

Strategy 3: Decompose by Data (Database Independence)

If two features share a table, they should be in the same service. If they can have separate databases, they are candidates for separate services.

💡 PRO TIP
The "strangler fig" pattern: Don't rewrite a monolith in one go.
Gradually strangle it by extracting one service at a time, routing traffic away.
Start with the most independently scalable or most frequently changed module.

2.2 Bounded Context — DDD Basics

A Bounded Context defines the scope of a domain model. The same word can mean different things in different contexts.

Concept	In E-commerce Context	In Support Context
"Customer"	Has cart, orders, payment methods	Has support tickets, chat history

Concept	In E-commerce Context	In Support Context
"Product"	Has price, stock, category	Has known issues, FAQs
"Order"	Is a purchase transaction	Is a support case

Each service should only own the part of "Customer" it needs. Do NOT share a single Customer object across all services — that creates tight coupling.

2.3 Inter-Service Communication: Sync vs Async

Type	When to Use	Technologies	Trade-offs
Synchronous (Request/Response)	Real-time data needed immediately (e.g., GET user profile)	REST, gRPC	Simple; creates coupling; cascading failures
Asynchronous (Event-Driven)	Actions that can happen in background (e.g., send welcome email after registration)	Kafka, RabbitMQ	Decoupled; resilient; harder to debug; eventual consistency

2.4 REST vs gRPC vs Messaging

Protocol	Format	Best For	Spring Support
REST (HTTP/1.1)	JSON	Public APIs, browser clients, CRUD ops	Spring MVC / WebFlux
gRPC (HTTP/2)	Protocol Buffers	Internal high-throughput service calls	Spring gRPC (Boot 3.x)
Kafka (async)	Avro / JSON	Event streaming, high-volume pipelines	Spring Kafka
RabbitMQ (async)	JSON / AMQP	Task queues, pub/sub messaging	Spring AMQP

💡 PRO TIP

Use REST for external APIs (developer-friendly, wide tooling support).

Use gRPC for internal service-to-service calls where performance matters (3-10x faster than REST).

Use Kafka for anything that needs to be replayed, audited, or consumed by multiple services.

⚡ QUICK REVISION

- Decompose by Business Capability or by DDD Bounded Context
- Each service = 1 bounded context = 1 database = 1 team
- Sync comm (REST/gRPC): when caller needs immediate response

- Async comm (Kafka/RabbitMQ): when decoupling and resilience matter more than immediacy
- Wrong decomposition = "distributed monolith" — worst of both worlds

⚙️ Section 3: Spring Boot Microservices Setup

Project structure, architecture, and foundational patterns

3.1 Recommended Project Structure

```
job-portal/
├── api-gateway/          # Spring Cloud Gateway
├── auth-service/         # JWT + OAuth2
├── user-service/         # User management
├── job-service/          # Job listings & applications
├── notification-service/ # Email/SMS via Kafka events
├── config-server/        # Spring Cloud Config
├── eureka-server/        # Service discovery
├── common-lib/           # Shared DTOs, exceptions, utils
└── docker-compose.yml    # Local dev environment
```

Each Service Follows This Internal Structure:

```
user-service/src/main/java/com/jobportal/user/
├── controller/           # REST endpoints (thin layer)
├── service/              # Business logic
├── repository/           # Data access (Spring Data JPA)
├── entity/               # JPA entities (@Entity)
├── dto/                  # Data Transfer Objects
├── mapper/               # Entity ↔ DTO conversion (MapStruct)
├── exception/            # Custom exceptions + global handler
├── config/               # Security, Kafka, Bean configs
└── UserServiceApplication.java
```

3.2 Maven Dependency Setup (Spring Boot 3.x)

```xml
<!-- pom.xml for a typical microservice -->
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
</parent>

<properties>
    <java.version>21</java.version>
    <spring-cloud.version>2023.0.1</spring-cloud.version>
</properties>

<dependencies>
    <!-- Core Web -->
    <dependency>spring-boot-starter-web</dependency>
    <!-- Data JPA -->
    <dependency>spring-boot-starter-data-jpa</dependency>
    <!-- Validation -->
    <dependency>spring-boot-starter-validation</dependency>
    <!-- Actuator (health, metrics) -->
    <dependency>spring-boot-starter-actuator</dependency>
    <!-- Eureka Client -->
    <dependency>spring-cloud-starter-netflix-eureka-client</dependency>
    <!-- Config Client -->
    <dependency>spring-cloud-starter-config</dependency>
    <!-- OpenFeign for REST calls -->
```

```
    <dependency>spring-cloud-starter-openfeign</dependency>
    <!-- Resilience4j -->
    <dependency>spring-cloud-starter-circuitbreaker-resilience4j</dependency>
    <!-- Kafka -->
    <dependency>spring-kafka</dependency>
    <!-- Security + JWT -->
    <dependency>spring-boot-starter-security</dependency>
    <dependency>jjwt-api:0.12.5</dependency>
    <!-- MapStruct -->
    <dependency>mapstruct:1.5.5.Final</dependency>
    <!-- Lombok -->
    <dependency>lombok</dependency>
    <!-- MySQL -->
    <dependency>mysql-connector-j</dependency>
</dependencies>
```

3.3 DTOs, Entities, and Mappers

Never expose JPA Entities directly from REST controllers. Always use DTOs.

Entity (JPA):

```
@Entity
@Table(name = "users")
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class UserEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(nullable = false)
    private String firstName;

    private String lastName;

    @Enumerated(EnumType.STRING)
    private UserRole role;

    @CreationTimestamp
    private LocalDateTime createdAt;
}
```

DTO (Response):

```
// Response DTO — what we expose to clients
@Data @Builder @NoArgsConstructor @AllArgsConstructor
public class UserResponseDto {
    private String id;
    private String email;
    private String firstName;
    private String lastName;
    private String role;
}

// Request DTO — what clients send us
```

```
@Data @NoArgsConstructor @AllArgsConstructor
public class CreateUserRequest {
    @NotBlank(message = "Email is required")
    @Email(message = "Invalid email format")
    private String email;

    @NotBlank @Size(min = 8)
    private String password;

    @NotBlank
    private String firstName;
}
```

MapStruct Mapper:

```
@Mapper(componentModel = "spring")
public interface UserMapper {

    UserResponseDto toDto(UserEntity entity);

    @Mapping(target = "id", ignore = true)
    @Mapping(target = "createdAt", ignore = true)
    UserEntity toEntity(CreateUserRequest request);

    List<UserResponseDto> toDtoList(List<UserEntity> entities);
}
```

3.4 Global Exception Handling

Never let raw exceptions leak from your API. Always return structured error responses.

```
// Custom Exception
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, String id) {
        super(resource + " not found with id: " + id);
    }
}

// Standard Error Response DTO
@Data @Builder
public class ApiError {
    private int status;
    private String message;
    private String path;
    private LocalDateTime timestamp;
    private Map<String, String> validationErrors;
}

// Global Exception Handler
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ApiError> handleNotFound(ResourceNotFoundException ex,
                                                   HttpServletRequest req) {
        return ResponseEntity.status(404).body(
            ApiError.builder()
                .status(404).message(ex.getMessage())
                .path(req.getRequestURI())
                .timestamp(LocalDateTime.now())
```

```
                .build()
        );
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex,
                                                     HttpServletRequest req) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
          .forEach(e -> errors.put(e.getField(), e.getDefaultMessage()));
        return ResponseEntity.badRequest().body(
            ApiError.builder().status(400).message("Validation failed")
                .validationErrors(errors).path(req.getRequestURI())
                .timestamp(LocalDateTime.now()).build()
        );
    }
}
```

⚡ QUICK REVISION
• Always use DTOs — never expose entities directly from REST APIs
• Use MapStruct for mapping (compile-time, no reflection, zero overhead)
• @RestControllerAdvice provides centralized exception handling
• Validation via @Valid on controller params + @NotBlank / @Email on DTOs
• UUID as primary key in microservices (avoids DB-level coordination)

🌐 Section 4: Service Communication

REST, Feign, Kafka, and resilience patterns

4.1 OpenFeign — Declarative REST Clients

OpenFeign lets you write HTTP client calls as simple Java interfaces — no boilerplate RestTemplate code.

Enable Feign:

```
@SpringBootApplication
@EnableFeignClients
public class JobServiceApplication { ... }
```

Define Feign Client:

```
@FeignClient(name = "user-service", fallback = UserClientFallback.class)
public interface UserServiceClient {

    @GetMapping("/api/users/{userId}")
    UserResponseDto getUserById(@PathVariable("userId") String userId);

    @PostMapping("/api/users/batch")
    List<UserResponseDto> getUsersByIds(@RequestBody List<String> userIds);
}

// Fallback — what to return when user-service is down
@Component
public class UserClientFallback implements UserServiceClient {

    @Override
    public UserResponseDto getUserById(String userId) {
        return UserResponseDto.builder().id(userId)
            .firstName("Unknown").build(); // graceful degradation
    }
}
```

4.2 Resilience4j — Circuit Breaker & Retry

Circuit Breaker is inspired by electrical circuit breakers — when too many calls fail, "open" the circuit to stop cascading failures.

State	Meaning	Behavior
CLOSED	Normal operation	Calls pass through normally
OPEN	Circuit tripped	All calls immediately fail/fallback (no network call)
HALF_OPEN	Testing if service recovered	Limited calls allowed to test recovery

Configuration (application.yml):

```
resilience4j:
  circuitbreaker:
    instances:
      user-service:
        sliding-window-size: 10
        failure-rate-threshold: 50       # Open if 50% of 10 calls fail
        wait-duration-in-open-state: 10s # Wait 10s before trying again
        permitted-number-of-calls-in-half-open-state: 3
  retry:
    instances:
      user-service:
        max-attempts: 3
        wait-duration: 500ms
        retry-exceptions:
          - feign.FeignException.ServiceUnavailable
```

Using in Service Layer:

```
@Service
public class JobService {

    private final UserServiceClient userClient;

    @CircuitBreaker(name = "user-service", fallbackMethod = "getUserFallback")
    @Retry(name = "user-service")
    public UserResponseDto getUser(String userId) {
        return userClient.getUserById(userId);
    }

    // Fallback must have same signature + Throwable param
    public UserResponseDto getUserFallback(String userId, Throwable ex) {
        log.warn("User service unavailable for userId: {}, error: {}", userId,
ex.getMessage());
        return UserResponseDto.builder().id(userId).firstName("Unavailable").build();
    }
}
```

4.3 Kafka — Async Event-Driven Communication

Apache Kafka is a distributed event streaming platform. Think of it as a highly durable, replayable message log.

Key Concepts:

- Topic: A named log/stream (like a database table for events)
- Partition: Topics are split into partitions for parallelism
- Producer: Service that publishes events
- Consumer: Service that reads and processes events
- Consumer Group: Multiple consumers sharing the work of one topic

Producer — Publish Event When User Registered:

```
// Event DTO (put in common-lib)
@Data @Builder @NoArgsConstructor @AllArgsConstructor
```

```
public class UserRegisteredEvent {
    private String userId;
    private String email;
    private String firstName;
    private Instant eventTime;
}

// Kafka Producer Service
@Service
@RequiredArgsConstructor
public class UserEventPublisher {

    private final KafkaTemplate<String, UserRegisteredEvent> kafkaTemplate;

    public void publishUserRegistered(UserRegisteredEvent event) {
        kafkaTemplate.send("user-registered", event.getUserId(), event)
            .whenComplete((result, ex) -> {
                if (ex != null) log.error("Failed to send event", ex);
                else log.info("Event sent: {}", result.getRecordMetadata());
            });
    }
}
```

Consumer — Notification Service Listens:

```
@Service
@RequiredArgsConstructor
public class NotificationConsumer {

    private final EmailService emailService;

    @KafkaListener(
        topics = "user-registered",
        groupId = "notification-service-group",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void handleUserRegistered(UserRegisteredEvent event,
                                     @Header(KafkaHeaders.RECEIVED_PARTITION) int
partition) {
        log.info("Received event from partition {}: {}", partition, event);
        emailService.sendWelcomeEmail(event.getEmail(), event.getFirstName());
    }
}
```

💡 PRO TIP
Always use a specific consumer group ID per service — never leave it as default.
Use String key = userId/orderId to ensure events for the same entity go to the same partition (ordering guarantee).
Enable idempotent producers (enable.idempotence=true) to prevent duplicate events.
Dead Letter Topics (DLT): Route failed messages to a -dlt topic for manual review.

⚡ QUICK REVISION
• OpenFeign: declarative HTTP client — define an interface, Spring handles the rest

• Circuit Breaker: CLOSED → OPEN → HALF_OPEN based on failure rate
• Fallback methods must match the original method signature + Throwable
• Kafka producer: kafkaTemplate.send(topic, key, value)
• Kafka consumer: @KafkaListener with groupId for each consuming service
• Always use key-based partitioning to maintain ordering for same entity

🧭 Section 5: Service Discovery & Registry

How services find each other in a dynamic environment

5.1 Why Service Discovery?

In microservices, service instances come and go dynamically (auto-scaling, crashes, deployments). You can't hardcode IPs. Service discovery solves this.

Pattern	How It Works	Example
Client-Side Discovery	Client queries registry, picks an instance, calls it directly	Eureka + Ribbon/Spring LB
Server-Side Discovery	Client calls load balancer, LB queries registry and routes	AWS ALB, Nginx, Consul + Envoy

5.2 Eureka Server Setup

eureka-server/pom.xml (dependency):

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

EurekaServerApplication.java:

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.yml:

```
server:
  port: 8761

eureka:
  instance:
    hostname: localhost
  client:
    register-with-eureka: false   # Server does not register itself
    fetch-registry: false         # Server does not fetch from others
  server:
    enable-self-preservation: false  # Dev only — disable in production
```

5.3 Registering a Microservice with Eureka

application.yml of user-service:

```yaml
spring:
  application:
    name: user-service   # This is the service name used for discovery

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka
    fetch-registry: true
    register-with-eureka: true
  instance:
    prefer-ip-address: true  # Important for Docker/Kubernetes
    instance-id: ${spring.application.name}:${random.value}
```

With this config, OpenFeign uses the service name "user-service" to look up available instances in Eureka and load balance automatically.

💡 PRO TIP
In production (Kubernetes), you typically do NOT need Eureka — K8s has built-in service discovery via kube-dns.
Consul is preferred over Eureka in production because it also does health checking and key-value config.
Always run Eureka in cluster mode (3+ instances) to avoid single point of failure.

⚡ QUICK REVISION
• Service registry = phonebook for microservices (name → IP:port)
• Eureka: Netflix OSS, easy Spring Boot integration, client-side discovery
• Register: spring.application.name + eureka.client.service-url
• Feign + Eureka = automatic client-side load balancing by service name
• In Kubernetes environments, native K8s DNS replaces Eureka

🚪 Section 6: API Gateway

The front door to your microservices

6.1 What is an API Gateway?

The API Gateway is the single entry point for all client requests. It handles cross-cutting concerns so individual services don't have to.

Functions of an API Gateway:

- Routing: Forward /api/users/** → user-service
- Authentication: Validate JWT before routing to services
- Rate Limiting: Limit clients to 100 requests/minute
- Load Balancing: Distribute across service instances
- Request/Response transformation
- SSL Termination: Decrypt HTTPS at gateway, use HTTP internally
- Logging & Tracing: Attach correlation IDs

6.2 Spring Cloud Gateway Setup

application.yml:

```yaml
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true         # Auto-discover routes from Eureka
          lower-case-service-id: true
      routes:
        - id: user-service
          uri: lb://user-service  # lb:// = load balanced via Eureka
          predicates:
            - Path=/api/users/**
          filters:
            - StripPrefix=0
            - name: CircuitBreaker
              args:
                name: user-service
                fallbackUri: forward:/fallback/users

        - id: job-service
          uri: lb://job-service
          predicates:
            - Path=/api/jobs/**
          filters:
            - StripPrefix=0
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
```

6.3 Authentication Filter at Gateway

Validate JWT tokens at the gateway — don't burden individual services with authentication logic.

```
@Component
public class JwtAuthenticationFilter implements GlobalFilter, Ordered {

    private static final List<String> OPEN_ENDPOINTS = List.of(
        "/api/auth/login", "/api/auth/register", "/api/auth/refresh"
    );

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String path = exchange.getRequest().getPath().value();

        // Skip auth for public endpoints
        if (OPEN_ENDPOINTS.stream().anyMatch(path::startsWith)) {
            return chain.filter(exchange);
        }

        String authHeader = exchange.getRequest()
            .getHeaders().getFirst(HttpHeaders.AUTHORIZATION);

        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }

        String token = authHeader.substring(7);
        try {
            Claims claims = jwtUtil.validateAndExtract(token);
            // Inject user info as headers for downstream services
            ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
                .header("X-User-Id", claims.getSubject())
                .header("X-User-Role", claims.get("role", String.class))
                .build();
            return chain.filter(exchange.mutate().request(mutatedRequest).build());
        } catch (JwtException e) {
            exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
            return exchange.getResponse().setComplete();
        }
    }

    @Override
    public int getOrder() { return -1; } // Run first
}
```

💡 PRO TIP
Never put business logic in the API Gateway — it should be a thin routing/security layer.
The gateway should pass user info (X-User-Id, X-User-Role) as request headers to downstream services.
Individual services trust these headers and do NOT re-validate JWTs (gateway already did that).

⚡ QUICK REVISION
• API Gateway = single entry point; handles routing, auth, rate limiting, logging
• Spring Cloud Gateway is non-blocking (Reactor/WebFlux based)
• uri: lb://service-name → load balanced via Eureka
• GlobalFilter: runs on every request; use for JWT validation
• Pass user context as headers (X-User-Id, X-User-Role) downstream

🔐 Section 7: Security

JWT, OAuth2, and securing microservices

7.1 JWT — JSON Web Token

JWT is a compact, self-contained token for securely transmitting claims between parties.

Part	Content	Example
Header	Algorithm and token type	{"alg":"HS256","typ":"JWT"}
Payload	Claims (user data, roles, expiry)	{"sub":"user123","role":"ADMIN","exp":1718000000}
Signature	HMAC or RSA of header.payload	HMACSHA256(base64(header)+"."+base64(payload), secret)

7.2 Auth Service — JWT Generation

```
// JwtUtil.java
@Component
public class JwtUtil {

    @Value("${jwt.secret}")
    private String secret;

    @Value("${jwt.expiration:86400000}")
    private long expirationMs;

    public String generateToken(UserDetails user, Map<String, Object> extraClaims) {
        return Jwts.builder()
            .claims(extraClaims)
            .subject(user.getUsername())
            .issuedAt(new Date())
            .expiration(new Date(System.currentTimeMillis() + expirationMs))
            .signWith(getSigningKey())
            .compact();
    }

    public Claims validateAndExtract(String token) {
        return Jwts.parser()
            .verifyWith(getSigningKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
    }
}
```

7.3 Auth Controller — Login & Register

```
@RestController
```

```
@RequestMapping("/api/auth")
@RequiredArgsConstructor
public class AuthController {

    private final AuthService authService;

    @PostMapping("/register")
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest req)
{
        return ResponseEntity.status(201).body(authService.register(req));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest req) {
        return ResponseEntity.ok(authService.login(req));
    }

    @PostMapping("/refresh")
    public ResponseEntity<AuthResponse> refresh(@RequestHeader("X-Refresh-Token") String
token) {
        return ResponseEntity.ok(authService.refreshToken(token));
    }
}

@Data @Builder
public class AuthResponse {
    private String accessToken;
    private String refreshToken;
    private long expiresIn;
    private UserResponseDto user;
}
```

7.4 Role-Based Access in Downstream Services

Downstream services trust the X-User-Id and X-User-Role headers set by the gateway.

```
// Security config in job-service (no JWT parsing needed!)
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(sm -> sm.sessionCreationPolicy(STATELESS))
            .addFilterBefore(headerAuthFilter(),
UsernamePasswordAuthenticationFilter.class)
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/jobs/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .build();
    }
}

// In Controller — use method-level security
@PreAuthorize("hasRole('ADMIN')")
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteJob(@PathVariable String id) { ... }
```

⚠️ COMMON MISTAKE / WARNING
NEVER store sensitive data (passwords, credit cards) in JWT payload — it's only base64 encoded, NOT encrypted.
ALWAYS use HTTPS in production — JWT in transit can be intercepted.
Use short expiry for access tokens (15-30 min) and longer for refresh tokens (7-30 days).
Rotate your JWT secret key regularly and support key rotation without downtime.

⚡ QUICK REVISION
• JWT = Header.Payload.Signature — self-contained, stateless auth
• Auth service generates tokens; API gateway validates them
• Downstream services trust X-User-Id/X-User-Role headers from gateway
• Use @PreAuthorize for method-level role-based access control
• Short-lived access token + long-lived refresh token pattern

🗄️ Section 8: Database per Service Pattern

Data isolation, Saga pattern, and transactions

8.1 Database per Service — Why It Matters

Each microservice should own its data. No shared databases between services.

Scenario	Shared DB (Anti-pattern)	Database per Service (Correct)
Schema changes	Break all services	Only affects the owning service
Scaling	Scale entire DB	Scale only the DB that needs it
Tech choice	All services use same DB	User service: MySQL, Job service: MongoDB
Team autonomy	DB team bottleneck	Each team manages their own DB
Data coupling	Services can JOIN across domains	Services must call each other's API

8.2 JPA Best Practices in Microservices

```
// Use FetchType.LAZY for all relationships
@OneToMany(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
@JoinColumn(name = "user_id")
private List<Application> applications;

// Always use @Version for optimistic locking
@Version
private Long version;  // prevents lost updates in concurrent scenarios

// Use Pageable for all list endpoints
@GetMapping
public Page<JobResponseDto> getAllJobs(
    @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "20") int size,
    @RequestParam(defaultValue = "createdAt") String sortBy) {
    return jobService.getAllJobs(PageRequest.of(page, size,
Sort.by(sortBy).descending()));
}
```

8.3 The Saga Pattern — Distributed Transactions

Traditional ACID transactions don't work across microservices. The Saga pattern breaks a distributed transaction into a series of local transactions with compensating actions on failure.

Example: Job Application Flow

- Step 1: Create Application (job-service) → Publish APPLICATION_CREATED event
- Step 2: Reserve Slot (job-service) → Publish SLOT_RESERVED event
- Step 3: Send Confirmation (notification) → Publish NOTIFICATION_SENT event
- On Failure: Compensate (cancel application, release slot)

Approach	How It Works	Pros	Cons
Choreography	Each service reacts to events from other services. No central coordinator.	Loose coupling, simple	Hard to trace, no clear workflow owner
Orchestration	A Saga orchestrator directs each step explicitly via commands/events.	Clear workflow, easy to monitor	Orchestrator is central point; can become complex

Choreography Saga — Example with Kafka:

```
// job-service: Publish event when application is created
kafkaTemplate.send("application-created",
    ApplicationCreatedEvent.builder()
        .applicationId(app.getId())
        .jobId(app.getJobId())
        .applicantId(app.getApplicantId())
        .status("PENDING")
        .build());

// notification-service: Listens and sends confirmation
@KafkaListener(topics = "application-created", groupId = "notification-group")
public void onApplicationCreated(ApplicationCreatedEvent event) {
    emailService.sendApplicationConfirmation(event.getApplicantId(), event.getJobId());
    // Then publish its own event
    kafkaTemplate.send("notification-sent", ...
}

// On failure: Publish compensating event
kafkaTemplate.send("application-cancelled",
    ApplicationCancelledEvent.builder()
        .applicationId(app.getId())
        .reason("Slot full")
        .build());
```

💡 PRO TIP

Use the Outbox Pattern to ensure events are published reliably.

Write the event to a local "outbox" table in the same DB transaction as the business data.

A separate poller reads from outbox and publishes to Kafka — guarantees no event loss.

⚡ QUICK REVISION

- Database per service = no shared schemas between microservices
- If services need the same data, call the owning service's API
- Saga = series of local transactions + compensating actions for rollback
- Choreography: event-driven, decentralized, services react to each other
- Orchestration: central Saga orchestrator directs each step
- Outbox pattern: write event to DB + publish to Kafka atomically

📡 Section 9: Configuration Management

Centralized config, profiles, and secrets

9.1 Spring Cloud Config Server

Store all microservice configurations in a central Git repository. Services fetch their config on startup.

config-server/application.yml:

```yaml
server:
  port: 8888

spring:
  application:
    name: config-server
  cloud:
    config:
      server:
        git:
          uri: https://github.com/your-org/microservices-config
          default-label: main
          search-paths: 'configs/{application}'
          clone-on-start: true
```

Config Repo Structure:

```
microservices-config/
├── configs/
│   ├── user-service/
│   │   ├── user-service.yml          # Default
│   │   ├── user-service-dev.yml      # Dev overrides
│   │   ├── user-service-staging.yml  # Staging overrides
│   │   └── user-service-prod.yml     # Prod overrides (no secrets here!)
│   ├── job-service/
│   └── api-gateway/
```

Client Service bootstrap.yml:

```yaml
spring:
  application:
    name: user-service
  config:
    import: "configserver:http://config-server:8888"
  profiles:
    active: ${SPRING_PROFILES_ACTIVE:dev}  # Controlled by env var
```

9.2 Secrets Management

⚠️ COMMON MISTAKE / WARNING
NEVER store secrets (DB passwords, JWT keys, API keys) in Git config repos.
In dev/testing: use environment variables.

In production: use HashiCorp Vault or AWS Secrets Manager.
Spring Cloud Vault integrates seamlessly with Spring Boot for secret injection.

```
# Production: Load secrets from environment variables
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}

jwt:
  secret: ${JWT_SECRET}
```

9.3 Config Refresh at Runtime

With @RefreshScope and the /actuator/refresh endpoint, services can reload config without restart.

```
@RestController
@RefreshScope  // Beans annotated with this are refreshed on /actuator/refresh call
public class FeatureFlagController {
    @Value("${feature.newJobAlgorithm.enabled:false}")
    private boolean newAlgorithmEnabled;
}

# Trigger config reload (webhook from Git can automate this):
POST http://user-service:8080/actuator/refresh
```

⚡ QUICK REVISION
• Config server stores all application configs in Git
• Services bootstrap: spring.config.import points to config server URL
• Profile-specific overrides: {appname}-{profile}.yml
• Secrets go in env vars or Vault — NEVER in Git
• @RefreshScope + /actuator/refresh = live config reload without restart

📊 Section 10: Logging, Tracing & Monitoring

Observability stack for production systems

10.1 Why Observability?

In a distributed system with 10+ services, a single request touches multiple services. Without centralized observability, debugging is impossible.

Pillar	What It Tells You	Tools
Logging	What happened (events, errors, state changes)	ELK Stack (Elasticsearch, Logstash, Kibana)
Tracing	How a request flowed across services (with timing)	Zipkin, Jaeger, OpenTelemetry
Metrics	System health over time (CPU, memory, request rates)	Prometheus + Grafana

10.2 Structured Logging with Logback + ELK

logback-spring.xml for JSON output:

```
<dependency>
    <groupId>net.logstash.logback</groupId>
    <artifactId>logstash-logback-encoder</artifactId>
    <version>7.4</version>
</dependency>
```

```
# application.yml — Always include correlation IDs in logs
logging:
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} [%X{traceId},%X{spanId}] %-5level %logger{36} -
%msg%n"

# Use SLF4J MDC to attach correlation IDs:
MDC.put("userId", userId);
MDC.put("requestId", UUID.randomUUID().toString());
log.info("Processing job application for user: {}", userId);
```

10.3 Distributed Tracing with Micrometer + Zipkin

```
<!-- Add to each service -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-brave</artifactId>
</dependency>
<dependency>
    <groupId>io.zipkin.reporter2</groupId>
    <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
```

```yaml
# application.yml
management:
  tracing:
    sampling:
      probability: 1.0    # 100% in dev; use 0.1 (10%) in production
  zipkin:
    tracing:
      endpoint: http://zipkin:9411/api/v2/spans
```

Every request now gets a traceId. You can search Zipkin to see the full call chain: Gateway → UserService → JobService → NotificationService with latencies at each hop.

10.4 Metrics with Prometheus + Grafana

```yaml
<!-- Expose Prometheus metrics via Actuator -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
  endpoint:
    health:
      show-details: always
```

Prometheus scrapes /actuator/prometheus every 15 seconds. Grafana visualizes these metrics. Key dashboards to set up:

- Request rate (RPS) per service
- Error rate (4xx, 5xx) per service
- P95/P99 latency per endpoint
- JVM heap usage, GC activity
- Kafka consumer lag (critical — indicates backpressure)

💡 PRO TIP
Set up alerts in Grafana: alert when error rate > 1% or P99 latency > 2000ms.
Use Micrometer's @Timed annotation to auto-instrument method execution times.
Log at INFO level for business events, DEBUG for technical details — never log in tight loops.

⚡ QUICK REVISION
• Logging (ELK): centralized, searchable, structured JSON logs with traceIds
• Tracing (Zipkin): follow a request across all services via traceId
• Metrics (Prometheus + Grafana): time-series dashboards and alerting
• Always include traceId in logs — makes cross-service debugging possible

- Sample 100% in dev, 10% in prod to reduce overhead

🐳 Section 11: Containerization & Deployment

Docker, Kubernetes, and CI/CD

11.1 Dockerizing a Spring Boot Service

Dockerfile (Multi-stage build — production ready):

```
# Stage 1: Build
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B   # Cache dependencies
COPY src ./src
RUN mvn package -DskipTests

# Stage 2: Run (minimal JRE image)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# Non-root user for security
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser

COPY --from=build /app/target/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java",
  "-XX:+UseContainerSupport",
  "-XX:MaxRAMPercentage=75.0",
  "-jar", "app.jar"]
```

11.2 Docker Compose — Local Development

```
version: '3.8'

services:
  eureka-server:
    build: ./eureka-server
    ports: ["8761:8761"]
    networks: [ms-network]

  config-server:
    build: ./config-server
    ports: ["8888:8888"]
    depends_on: [eureka-server]
    networks: [ms-network]

  api-gateway:
    build: ./api-gateway
    ports: ["8080:8080"]
    depends_on: [eureka-server, config-server]
    environment:
      - SPRING_PROFILES_ACTIVE=dev
    networks: [ms-network]

  user-service:
```

```
    build: ./user-service
    depends_on: [mysql, eureka-server, kafka]
    environment:
      - SPRING_PROFILES_ACTIVE=dev
      - DB_URL=jdbc:mysql://mysql:3306/userdb
      - DB_USERNAME=root
      - DB_PASSWORD=secret
    networks: [ms-network]

  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: secret
      MYSQL_DATABASE: userdb
    volumes: [mysql-data:/var/lib/mysql]
    networks: [ms-network]

  kafka:
    image: confluentinc/cp-kafka:7.6.0
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    depends_on: [zookeeper]
    networks: [ms-network]

  zookeeper:
    image: confluentinc/cp-zookeeper:7.6.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
    networks: [ms-network]

  zipkin:
    image: openzipkin/zipkin
    ports: ["9411:9411"]
    networks: [ms-network]

networks:
  ms-network:
    driver: bridge

volumes:
  mysql-data:
```

11.3 Kubernetes Basics

Kubernetes (K8s) orchestrates containerized applications — it replaces Docker Compose in production.

K8s Object	Purpose	Analogy
Pod	Smallest deployable unit — runs 1+ containers	A running container (like a running Docker container)
Deployment	Manages Pod replicas, rolling updates, rollbacks	A recipe for how many pods to run
Service	Stable network endpoint to reach pods (load balanced)	A load balancer in front of pods
ConfigMap	Non-sensitive config key-value pairs	Config file mounted into pods

K8s Object	Purpose	Analogy
Secret	Sensitive data (passwords, keys) base64 encoded	Encrypted config for secrets
Ingress	HTTP routing rules, acts as API gateway at K8s level	Nginx reverse proxy for K8s
HPA	Horizontal Pod Autoscaler — scale pods based on CPU	Auto-scaling based on traffic

Kubernetes Deployment for user-service:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: user-service
  namespace: job-portal
spec:
  replicas: 3
  selector:
    matchLabels:
      app: user-service
  template:
    metadata:
      labels:
        app: user-service
    spec:
      containers:
      - name: user-service
        image: your-registry/user-service:1.2.3
        ports:
        - containerPort: 8080
        env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: user-service-secrets
              key: db-password
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 20
          periodSeconds: 5
```

11.4 CI/CD with GitHub Actions

```yaml
# .github/workflows/user-service.yml
name: User Service CI/CD

on:
  push:
    branches: [main]
    paths: ['user-service/**']

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Java 21
        uses: actions/setup-java@v4
        with:
          java-version: '21'
          distribution: temurin

      - name: Build and Test
        run: mvn clean verify
        working-directory: user-service

      - name: Build Docker Image
        run: |
          docker build -t ${{ secrets.REGISTRY }}/user-service:${{ github.sha }} .
          docker push ${{ secrets.REGISTRY }}/user-service:${{ github.sha }}
        working-directory: user-service

      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/user-service \
            user-service=${{ secrets.REGISTRY }}/user-service:${{ github.sha }} \
            -n job-portal
```

⚡ QUICK REVISION
• Multi-stage Dockerfile: separate build and runtime for minimal image size
• Run containers as non-root user for security
• Docker Compose for local dev; Kubernetes for production
• Always set K8s resource requests/limits to prevent resource starvation
• liveness probe = is app alive? readiness probe = is app ready for traffic?
• CI/CD: build → test → containerize → push to registry → deploy to K8s

⚡ Section 12: Performance & Scalability

Caching, load balancing, and horizontal scaling

12.1 Redis Caching

Cache frequently read, rarely changing data to reduce database load and response times.

```yaml
# application.yml
spring:
  data:
    redis:
      host: ${REDIS_HOST:localhost}
      port: 6379
  cache:
    type: redis
    redis:
      time-to-live: 3600000  # 1 hour
```

```java
@Service
@RequiredArgsConstructor
public class JobService {

    // Method-level caching with Spring Cache abstraction
    @Cacheable(value = "jobs", key = "#jobId",
               unless = "#result == null")
    public JobResponseDto getJobById(String jobId) {
        return jobRepository.findById(jobId)
            .map(jobMapper::toDto)
            .orElseThrow(() -> new ResourceNotFoundException("Job", jobId));
    }

    // Evict cache when job is updated
    @CacheEvict(value = "jobs", key = "#jobId")
    public JobResponseDto updateJob(String jobId, UpdateJobRequest req) {
        // ... update logic
    }

    // Evict ALL entries in cache
    @CacheEvict(value = "jobs", allEntries = true)
    public void refreshAllJobs() { ... }
}
```

⚠️ COMMON MISTAKE / WARNING
Cache invalidation is one of the hardest problems in CS. Always think about when to evict.
Do NOT cache user-specific or sensitive data without careful TTL design.
Implement a cache-aside pattern for complex scenarios (try cache → miss → DB → populate cache).

12.2 Horizontal Scaling Patterns

Strategy	What It Does	When to Use
Stateless services	No session state in app; state in	Always — prerequisite for horizontal

Strategy	What It Does	When to Use
	DB/Redis	scaling
Load balancing	Route traffic across multiple instances	When single instance is the bottleneck
Database read replicas	Read from replicas, write to primary	Read-heavy workloads (most apps)
CQRS	Separate read and write models/databases	High read/write ratio disparity
Async processing	Handle spikes via queues (Kafka), process at your pace	Bursty traffic (e.g., Black Friday)

12.3 Database Optimization

- Always index columns used in WHERE, JOIN, and ORDER BY clauses
- Use connection pooling (HikariCP, default in Spring Boot) — never create connections per request
- For read-heavy: enable @Transactional(readOnly = true) — uses read replicas if configured
- Paginate large result sets — never return unbounded lists from APIs
- Use @BatchSize for collections to prevent N+1 query problem

💡 PRO TIP

N+1 Query Problem: If you load 100 jobs and then query each job's applications separately, that's 101 queries.

Solution: Use @EntityGraph or JOIN FETCH in JPQL to load associations eagerly when needed.

Enable Hibernate query statistics in dev (spring.jpa.properties.hibernate.generate_statistics=true) to catch N+1 early.

⚡ QUICK REVISION

- Redis: @Cacheable / @CacheEvict / @CachePut annotations
- Cache-aside: try cache → miss → DB → store in cache
- Stateless services = prerequisite for horizontal scaling
- Database: index joins/where clauses, use connection pools, paginate everything
- @Transactional(readOnly = true) enables read replica routing

🧪 Section 13: Testing Strategy

Unit, integration, and contract testing

13.1 Testing Pyramid

Level	Speed	Coverage Goal	Tools
Unit Tests	Fastest (ms)	70–80% of all tests	JUnit 5, Mockito
Integration Tests	Slower (s)	15–20%	Spring Boot Test, Testcontainers
Contract Tests	Medium	5–10%	Spring Cloud Contract, Pact
E2E Tests	Slowest (min)	Few key user flows	REST Assured, Cypress

13.2 Unit Testing with JUnit 5 + Mockito

```
@ExtendWith(MockitoExtension.class)
class JobServiceTest {

    @Mock
    private JobRepository jobRepository;

    @Mock
    private JobMapper jobMapper;

    @InjectMocks
    private JobService jobService;

    @Test
    @DisplayName("Should return job when valid ID is provided")
    void getJobById_validId_returnsJob() {
        // Arrange
        String jobId = "job-123";
        JobEntity entity = JobEntity.builder().id(jobId).title("Java Dev").build();
        JobResponseDto dto = JobResponseDto.builder().id(jobId).title("Java
Dev").build();

        when(jobRepository.findById(jobId)).thenReturn(Optional.of(entity));
        when(jobMapper.toDto(entity)).thenReturn(dto);

        // Act
        JobResponseDto result = jobService.getJobById(jobId);

        // Assert
        assertThat(result.getId()).isEqualTo(jobId);
        assertThat(result.getTitle()).isEqualTo("Java Dev");
        verify(jobRepository, times(1)).findById(jobId);
    }

    @Test
    @DisplayName("Should throw ResourceNotFoundException when job not found")
    void getJobById_invalidId_throwsException() {
        when(jobRepository.findById(any())).thenReturn(Optional.empty());
        assertThatThrownBy(() -> jobService.getJobById("bad-id"))
```

```
            .isInstanceOf(ResourceNotFoundException.class)
            .hasMessageContaining("Job");
    }
}
```

13.3 Integration Testing with Testcontainers

Testcontainers spins up real Docker containers (MySQL, Kafka, Redis) for integration tests.

```
@SpringBootTest
@Testcontainers
@AutoConfigureMockMvc
class UserControllerIntegrationTest {

    @Container
    static MySQLContainer<?> mysql = new MySQLContainer<>("mysql:8.0")
        .withDatabaseName("testdb")
        .withUsername("test")
        .withPassword("test");

    @DynamicPropertySource
    static void configureProperties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", mysql::getJdbcUrl);
        registry.add("spring.datasource.username", mysql::getUsername);
        registry.add("spring.datasource.password", mysql::getPassword);
    }

    @Autowired MockMvc mockMvc;

    @Test
    void createUser_validRequest_returns201() throws Exception {
        String requestBody = """
            {
              "email": "test@example.com",
              "password": "SecurePass123",
              "firstName": "John"
            }
            """;

        mockMvc.perform(post("/api/users")
            .contentType(APPLICATION_JSON)
            .content(requestBody))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.email").value("test@example.com"))
            .andExpect(jsonPath("$.id").isNotEmpty());
    }
}
```

⚡ QUICK REVISION
• 70% unit → 20% integration → 10% contract/E2E testing ratio
• Mock dependencies with @Mock; test class under test with @InjectMocks
• Testcontainers: real DB in Docker for integration tests — no mocking the DB
• Use @DynamicPropertySource to inject Testcontainer connection details
• Name tests clearly: methodUnderTest_scenario_expectedBehavior

🏗️ Section 14: Real-World Project Walkthrough

Job Portal — end-to-end microservices system

14.1 System Overview — Job Portal

Services and their responsibilities:

Service	Responsibility	DB	Port
api-gateway	Routing, JWT validation, rate limiting	None	8080
auth-service	Login, register, token generation/refresh	MySQL	8081
user-service	User profiles, recruiter/candidate mgmt	MySQL	8082
job-service	Job listings, search, applications	PostgreSQL	8083
notification-service	Email/SMS via Kafka events	None	8084
config-server	Centralized configuration from Git	None	8888
eureka-server	Service discovery registry	None	8761

14.2 Key Flows

Flow 1: User Registration

1. POST /api/auth/register → api-gateway
2. Gateway routes to auth-service
3. auth-service creates user in DB, generates JWT
4. auth-service publishes USER_REGISTERED event to Kafka
5. notification-service consumes event, sends welcome email
6. Returns AuthResponse with tokens

Flow 2: Job Application

7. POST /api/jobs/{id}/apply → api-gateway
8. Gateway validates JWT, injects X-User-Id header
9. job-service receives request, checks application eligibility
10. job-service fetches user details via Feign → user-service
11. job-service saves application, publishes APPLICATION_CREATED event
12. notification-service sends confirmation email to candidate
13. notification-service sends notification email to recruiter

14.3 Key Design Decisions

- Auth service owns the "identity" — user-service owns the "profile"
- Job service calls user-service synchronously (Feign) for immediate profile data
- Notifications are always async via Kafka — user doesn't wait for email delivery
- All inter-service calls go through service names (Eureka), not IPs
- API Gateway is the only public-facing component — all services are internal

14.4 GitHub Project Structure

```
job-portal-microservices/
├── README.md
├── docker-compose.yml
├── docker-compose.override.yml    # Local dev overrides
├── k8s/
│   ├── namespace.yml
│   ├── secrets.yml
│   ├── user-service/
│   │   ├── deployment.yml
│   │   ├── service.yml
│   │   └── hpa.yml
│   └── ...
├── common-lib/
│   └── src/main/java/com/jobportal/common/
│       ├── dto/
│       ├── events/
│       └── exception/
├── api-gateway/
├── auth-service/
├── user-service/
├── job-service/
├── notification-service/
├── config-server/
├── eureka-server/
└── .github/
    └── workflows/
        ├── user-service.yml
        └── job-service.yml
```

💡 PRO TIP
Use semantic versioning for services: 1.2.3 (major.minor.patch).
Tag Docker images with git commit SHA for exact traceability.
Common-lib should be versioned and published to Nexus/Artifactory — not copy-pasted.

❓ Section 15: Interview Preparation

50+ questions with detailed answers

15.1 Core Concept Questions

Q1: What are microservices and how do they differ from monoliths?

Microservices is an architectural style that structures an application as a collection of small, independently deployable services organized around business capabilities. Each service has its own process, data store, and is deployed independently.

Key differences: Monoliths deploy everything together, share a single database, and scale as a unit. Microservices deploy independently, own their data, scale individual services, and allow different technology stacks per service.

Q2: How would you decompose a monolith into microservices?

Use the strangler fig pattern: identify bounded contexts using DDD, start with the most independently scalable component, route traffic gradually using a proxy, ensure the new service has its own database before extracting, validate with feature flags before cutting over fully.

Q3: What is the Circuit Breaker pattern and why is it needed?

Without circuit breakers, a slow downstream service causes the calling service's threads to block indefinitely, consuming all resources and causing cascading failures across the system. Circuit breakers stop calling a failing service by "opening" the circuit, return immediately with a fallback, and periodically test if the service has recovered (half-open state).

Q4: How does distributed transaction management work in microservices?

Traditional 2PC (two-phase commit) is impractical across microservices due to tight coupling and blocking nature. Instead, use the Saga pattern: either choreography (services react to events) or orchestration (central saga orchestrator coordinates steps). Each step is a local transaction; compensating transactions handle rollbacks. Eventual consistency is accepted.

Q5: How would you handle inter-service communication failures?

Multiple layers of defense: Retry (3 attempts with exponential backoff), Circuit Breaker (stop calling a failing service), Fallback (return cached/default data), Timeout (don't wait forever), Bulkhead (isolate thread pools per service to prevent resource exhaustion).

15.2 Design & Architecture Questions

Q6: Design a rate limiter for an API Gateway

Use the Token Bucket algorithm backed by Redis. Each client gets a "bucket" with a max capacity. Tokens refill at a fixed rate. Each request costs one token. If bucket is empty, return 429 Too Many Requests. Spring Cloud Gateway has built-in RedisRateLimiter. In Redis, use INCR + TTL for atomic counter, or a Lua script for token bucket.

Q7: How would you ensure no message is lost in Kafka?

Producer: enable.idempotence=true, acks=all, retries=MAX_INT. Consumer: disable auto-commit, commit offsets only after successful processing. Use Dead Letter Topics for messages that fail after max retries. Kafka topic: replication.factor >= 3, min.insync.replicas=2. Monitor consumer lag for backpressure signals.

Q8: How do you handle service-to-service authentication?

Option 1: JWT propagation — forward the user's JWT to downstream services. Option 2: Service accounts — each service has its own JWT/mTLS credential for machine-to-machine auth. Option 3: mTLS — mutual TLS where both client and server present certificates (used in service meshes like Istio). In simple setups, trusting gateway-injected headers (X-User-Id) with network-level security is common.

Q9: Explain the Outbox Pattern

Problem: You want to save data to DB AND publish a Kafka event atomically — but DB and Kafka are separate systems, so you can't use one transaction. Solution: Write the event to an "outbox" table in the same DB transaction as the business data. A separate Outbox Processor reads unpublished events and publishes them to Kafka, then marks them as published. Guarantees at-least-once delivery.

Q10: What is CQRS?

Command Query Responsibility Segregation: separate the write model (commands) from the read model (queries). Write side handles business rules and writes to a "write database." An event is published and a projection updates the "read database" optimized for queries. Allows independent scaling of reads/writes and optimized read models (e.g., denormalized for fast dashboards).

15.3 Scenario-Based Questions

Q11: Your order-service is getting 5000 requests/sec but user-service can only handle 1000/sec. How do you handle this?

First, add Circuit Breaker on order-service's user-service calls. Cache user data in Redis with appropriate TTL (user profile changes rarely). For non-critical user data, serve from cache even if stale. For critical data, implement a local in-memory cache with short TTL. Scale user-service horizontally. If user data is needed for order creation, consider embedding minimal user data in the order service's own store (denormalization for resilience).

Q12: How would you debug a latency spike in a system with 15 microservices?

Start with distributed tracing (Zipkin/Jaeger): search by traceId or high-latency requests to find the slow service and hop. Check metrics (Prometheus/Grafana) for the identified service: CPU, memory, GC pauses, DB connection pool exhaustion. Check Kafka consumer lag if async. Look at centralized logs (Kibana) filtered by service and time window. Check slow query logs in the DB.

Q13: How do you do zero-downtime deployment?

Use rolling deployments in Kubernetes (default). New pods start, health checks pass, then old pods are terminated. Use readiness probes to ensure no traffic is sent before service is ready. For database migrations: use backward-compatible migrations only (add columns, never rename/drop). Use feature flags to enable new code paths after deployment. Blue-green deployment: switch traffic from v1 to v2 all at once — easier rollback.

15.4 System Design Questions

Q14: Design the notification system for a food delivery app

Services publish domain events: ORDER_PLACED, DRIVER_ASSIGNED, ORDER_DELIVERED. Notification service consumes these events from Kafka. Uses template engine for messages. Sends via email (SES), SMS (Twilio), push (Firebase) based on user preferences stored in its own DB. Retry with exponential backoff for transient failures. Dead letter topic for permanent failures. Rate limit: max 1 SMS/minute/user to prevent spam.

Q15: How would you implement global search across multiple microservices?

Do NOT query each service's database — this creates coupling and performance issues. Use CQRS/event sourcing: each service publishes change events to Kafka. A search indexer service consumes events and writes to Elasticsearch. Client queries Elasticsearch directly for search. Service events are the source of truth; Elasticsearch is a read-optimized projection. Elasticsearch handles full-text, faceted, and geo-search across all service data.

15.5 Quick-Fire Questions

Question	Answer
What is service mesh?	Infrastructure layer for service-to-service comms (Istio/Linkerd) — handles mTLS, tracing, circuit breaking at network level
What is an idempotent API?	Same request can be made multiple times with same result (PUT is idempotent; POST is not). Use idempotency keys for payments.
What is eventual consistency?	Data will be consistent across services eventually (not immediately). Acceptable trade-off in distributed systems for availability.
API versioning strategies?	URI versioning (/v1/users), Header versioning (Accept: application/vnd.api+json;version=2), Query param (?version=2)

Question	Answer
What is a sidecar pattern?	Deploy a helper container alongside main service (Envoy proxy, log shipper, config reloader) without changing app code.
Health check vs readiness?	Liveness: is app alive? If no, restart it. Readiness: is app ready for traffic? If no, remove from load balancer.
What is fan-out/fan-in?	Fan-out: one request triggers multiple parallel downstream calls. Fan-in: aggregate results. Use CompletableFuture or WebClient.
What is a BFF?	Backend For Frontend — separate API gateway tailored for each frontend (mobile BFF, web BFF) with different response shapes.
How do you handle schema evolution in Kafka?	Use Avro with Confluent Schema Registry. Supports backward/forward compatible evolution with schema versioning.
What is a bulkhead pattern?	Isolate thread pools/connections per downstream service. Failure in one bulkhead doesn't exhaust resources for others.

⚡ QUICK REVISION

- Circuit Breaker states: CLOSED (normal) → OPEN (failing) → HALF_OPEN (testing)
- Saga: compensating transactions for distributed rollback
- Outbox pattern: atomic DB write + Kafka publish guarantee
- CQRS: separate read and write models for independent optimization
- Zero-downtime: rolling deploy + readiness probe + backward-compatible DB migrations

⚠️ Section 16: Common Mistakes & Best Practices

Pitfalls from real production systems

16.1 Anti-Patterns to Avoid

1. Distributed Monolith

Splitting a monolith into services that are so tightly coupled they must be deployed together. Signs: services share a database, services can't run independently, all services must change together for any feature.

2. Over-granular Services

Creating too many tiny services (nanoservices). If a "service" is just one class with one endpoint, it's too small. Start with larger services and split only when there's a clear, justified need.

3. Synchronous Chains

ServiceA calls ServiceB which calls ServiceC which calls ServiceD. One slow/down service breaks the chain. Solution: use async messaging where possible; use circuit breakers everywhere; consider aggregating calls.

4. Ignoring the Fallacies of Distributed Computing

New developers assume: the network is reliable, latency is zero, bandwidth is infinite, the network is secure. Design for all of these to be false.

5. No Idempotency

Retries + non-idempotent operations = duplicate orders, duplicate charges. Use idempotency keys for all mutating operations. If you receive the same idempotency key twice, return the original response.

16.2 Production Checklist

Category	Checklist Items
Security	☐ JWT with short expiry ☐ HTTPS everywhere ☐ No secrets in code ☐ Input validation ☐ Non-root Docker
Resilience	☐ Circuit breakers ☐ Retries with backoff ☐ Timeouts set ☐ Fallbacks defined ☐ Bulkheads
Observability	☐ Structured JSON logs ☐ Distributed tracing ☐ Health endpoints ☐ Metrics + alerts ☐ Correlation IDs

Category	Checklist Items
Data	☐ DB per service ☐ Migrations scripts (Flyway) ☐ Connection pool configured ☐ Outbox for events
Deployment	☐ Docker multi-stage ☐ Readiness probes ☐ Resource limits ☐ Rolling deploy ☐ Rollback plan
Performance	☐ Redis caching ☐ Async where possible ☐ Pagination on lists ☐ DB indexes ☐ Connection pooling

16.3 Best Practices Summary

- Design for failure: every remote call can fail
- Make services stateless: store state in Redis/DB, not in-memory
- Version your APIs: never break consumers without deprecation notice
- Document with OpenAPI/Swagger: auto-generate from Spring Boot
- Use correlation IDs: trace requests across all services
- Test in production with chaos: Netflix's Chaos Monkey kills random services
- Single responsibility: one service, one bounded context, one team
- Prefer async: events over synchronous chains when possible
- Monitor everything: you cannot fix what you cannot see

💡 PRO TIP
The "three amigos" of microservices success: strong DevOps culture, automated testing, and domain expertise.
Don't start with 15 services. Start with 3-4 well-defined services and grow organically based on actual pain points.
"If in doubt, don't split" — a modular monolith beats a poorly-designed microservices system every time.

🧠 Section 17: How to Think in Interviews

Frameworks for system design

17.1 The IDEAL Framework for Microservices Design

Letter	Step	Questions to Ask
I	Identify requirements	Functional + non-functional? Scale targets? Read/write ratio? Latency SLA?
D	Define boundaries	What are the bounded contexts? What does each service own?
E	Establish communication	What needs to be sync vs async? Which calls need strong consistency?
A	Address cross-cutting	Auth? Config? Logging? Tracing? How to handle failure?
L	Layout infrastructure	DB choices? Kafka? Gateway? K8s? CI/CD? Cache?

17.2 Answering System Design Questions

Step-by-step approach for a 45-minute interview:

14. Clarify requirements (3 min): scale, users, key features, SLAs
15. High-level design (5 min): draw services, databases, message queues
16. Deep dive requested component (15 min): API design, DB schema, key algorithms
17. Address non-functionals (10 min): scaling, caching, failure handling
18. Trade-offs discussion (7 min): why these choices vs alternatives
19. Handle follow-ups (5 min): additional scenarios interviewer raises

17.3 Decision-Making Framework

Should this be its own service?

Ask: Does it have different scaling requirements? Does it change independently from other code? Does it have clear ownership? Does it have a separate data model? Is it used by multiple other services? If 3+ yes answers → extract it.

Should this be sync or async?

Use sync (REST/gRPC) when: the caller needs the response to continue, strong consistency is required, operation is user-facing with low latency requirement. Use async (Kafka/RabbitMQ) when: caller can continue without response, operation is fire-and-forget, multiple services need to react, failure handling and retry are important.

SQL or NoSQL?

SQL (MySQL/PostgreSQL): complex queries, transactions, relational data, ACID compliance. NoSQL (MongoDB): flexible schema, document-oriented data. Redis: caching, sessions, rate limiting. Cassandra: time-series, write-heavy, no complex queries. Elasticsearch: full-text search, aggregations.

💡 PRO TIP
Interviewers want to see your thought process, not just the answer.
Always state trade-offs: "I'm choosing X because Y, the trade-off is Z."
Reference real-world companies: "Netflix uses Hystrix/Resilience4j for this reason..."
Draw diagrams: visuals show architectural thinking and make complex explanations clear.

⚡ QUICK REVISION
• IDEAL: Identify → Define → Establish comm → Address cross-cutting → Layout infra
• Always clarify scale requirements first — 1K vs 1M users changes the design
• State trade-offs explicitly — shows senior-level thinking
• Sync = needs immediate response; Async = decoupled, eventual consistency OK
• 3+ yes to service extraction criteria = extract the service